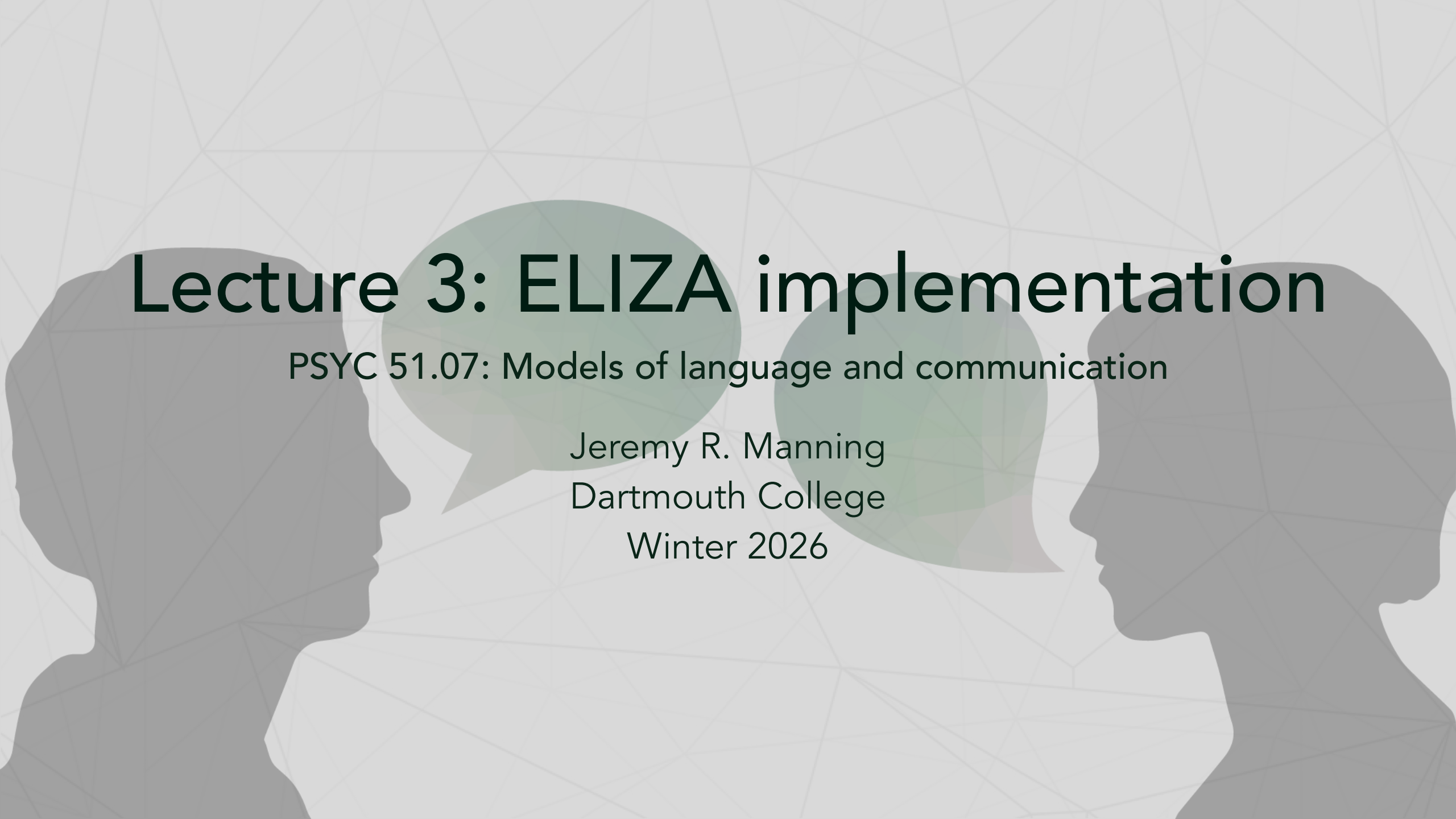




Lecture 3: ELIZA implementation

PSYC 51.07: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Today's agenda

What we'll cover

1. **Demo:** See ELIZA in action
2. **Algorithm:** How ELIZA actually works
3. **Code:** Key implementation patterns
4. **Vibe coding:** Tools and best practices
5. **Assignment 1:** Getting started

Goal

Leave today knowing *exactly* how to build your own ELIZA.

Let's try ELIZA

Interactive demo

ELIZA Demo

Try these inputs:

- "I am feeling sad"
- "My mother never understood me"
- "I think you are just a computer"

Exercise

Before I reveal how it works: can you guess what rules are being used?

The ELIZA algorithm



The complete ELIZA processing pipeline

Key insight

Each step is simple string manipulation. No "understanding" required!

Step 1: Pre-substitutions

Normalize input before pattern matching.

```
1  pre_subs = {  
2      "dont": "don't",  
3      "cant": "can't",  
4      "im": "I'm",  
5      "youre": "you're",  
6      "recollect": "remember"  
7  }  
8  
9  def apply_pre_subs(text, subs):  
10     for old, new in subs.items():  
11         text = text.replace(old, new)
```

Why pre-substitutions?

Users type in many ways. Normalize first, then match patterns.

Step 2: Pattern matching

Find the best matching rule for the input.

```
1  rules = [  
2      {"pattern": r"(.*) my (mother|father|family) (.*)",  
3       "rank": 100},  
4      {"pattern": r"I am (.*)",  
5       "rank": 50},  
6      {"pattern": r"(.*)",  
7       "rank": 0}  # Fallback  
8  ]
```

Priority matters

Higher rank = higher priority. Check specific patterns before general ones.

Step 3: Decomposition

Capture groups extract the important parts.

```
1  import re
2
3  pattern = r"I am (.*)"
4  text = "I am feeling sad"
5
6  match = re.search(pattern, text)
7  if match:
8      captured = match.group(1)  # "feeling sad"
```

Regular expressions

The `(.*)` captures "anything" into a group we can use later.

Step 4: Reassembly

Insert captured content into response templates.

```
1 templates = [  
2     "Why are you {1}?",  
3     "How long have you been {1}?",  
4     "What made you {1}?"  
5 ]  
6  
7 # "feeling sad" → "Why are you feeling sad?"  
8 response = random.choice(templates).format(captured)
```

Variety

Multiple templates prevent repetitive responses.

Step 5: Post-substitutions

Flip pronouns for grammatical responses.

```
1  post_subs = {  
2      " i ": " you ",  
3      " my ": " your ",  
4      " am ": " are ",  
5      " me ": " you ",  
6      " I ": " you "  
7  }  
8  
9  # "Tell me about your mother"  
10 # → "Tell you about your mother" (fixed pronouns)
```

Tricky

Order and spacing matter. Be careful with word boundaries.

Worked example: Complete ELIZA pipeline

Input: "I am worried about my mother"

Step 1: Pre-subs (normalize input)

```
1 | "I am worried about my mother"  
2 | → No changes needed
```

Step 2: Pattern match

```
1 | Pattern: r"(.*) my (mother|father)  
   (.*)"   
2 | Rank: 100 (high priority - family!)   
3 | MATCH!
```

Step 3: Decompose (extract groups)

```
1 | Group 1: "I am worried about"  
2 | Group 2: "mother"  
3 | Group 3: "" (empty)
```

Step 4: Reassemble

```
1 | Template: "Tell me more about your  
   {2}."  
2 | → "Tell me more about your mother."
```

Step 5: Post-subs (fix pronouns)

```
1 | "Tell me more about your mother."  
2 | → No changes needed (no "I", "my",  
   etc.)
```

Final output:

```
1 | "Tell me more about your mother."
```

Worked example: Pronoun substitution

Input: "I think you hate me"

Pattern matched:

1	<code>r"(.*) you (.*) me"</code>
2	Template: "What makes you think I {2} you?"

Post-substitution fixes this:

1	Original captured: "hate"
2	In template: "I {2} you"
3	→ "I hate you"

Putting it together

```
1  def eliza_respond(user_input):
2      # 1. Normalize input
3      text = apply_pre_subs(user_input, pre_subs)
4
5      # 2. Find matching rule (highest rank first)
6      rule = find_best_match(text, rules)
7
8      # 3. Decompose: extract captured groups
9      groups = decompose(text, rule["pattern"])
10
11     # 4. Reassemble: build response
12     response = reassemble(groups, rule["templates"])
13
14     # 5. Fix pronouns
```

Vibe coding: AI-assisted development

Dartmouth AI Tools

- ai-tools.dartmouth.edu
- Free access to Claude, GPT-4, Gemini
- Dartmouth login required

Other options

- [ChatGPT](#)
- [Claude](#)
- [Gemini](#)
- GitHub Copilot

Recommended IDEs

For this course

Google Colaboratory (recommended)

- Free, runs in browser
- Easy to share notebooks
- GPU access when needed

For advanced users

- VS Code with Copilot extension
- Cursor - AI-first editor
- Antigravity - Python-focused

Effective prompting

Start with a plan

Before coding, describe what you want to build. Be specific about inputs, outputs, and behavior.

Good prompt example

"I'm building an ELIZA chatbot in Python. I need a function that takes a user input string, applies pre-substitutions from a dictionary, then matches against a list of regex patterns sorted by rank. Show me how to structure this."

Prompting best practices

1. **Be specific** about what you want
2. **Provide context** about your project
3. **Show examples** of input/output
4. **Iterate** on the response
5. **Test incrementally** as you build

Important

Don't expect perfect code on the first try. Refine your prompts based on what you get.

Testing strategies

Test incrementally

1. Test pre-substitutions alone
2. Test pattern matching alone
3. Test full pipeline with simple inputs
4. Test edge cases

Edge cases to consider:

- Empty input
- Very long input
- Special characters (!@#\$%)
- No pattern matches
- Multiple patterns could match

Worked example: Testing edge cases

```
1  # Test 1: Empty input
2  >>> eliza_respond("")
3  "I see." # Fallback response
4
5  # Test 2: Special characters
6  >>> eliza_respond("I am @#$%^& confused!")
7  "Why are you @#$%^& confused?" # Copies them verbatim
8
9  # Test 3: Multiple patterns match
10 >>> eliza_respond("I am sad about my mother")
11 # Pattern 1: "I am (.*)" → rank 50
12 # Pattern 2: "(.*) my mother (.*)" → rank 100
13 # Winner: Pattern 2 (higher rank)!
14 "Tell me more about your mother."
15
16 # Test 4: Greedy matching problem
17 >>> pattern = r"I am (.*)"
18 >>> text = "I am happy. I am sad."
19 >>> re.search(pattern, text).group(1)
20 "happy. I am sad " # Matched too much!
```

continued...

Worked example: Testing edge cases

```
21  
22 # Fix: Use non-greedy  
23 >>> pattern = r"I am (.*)\." # .*? stops at first .
```

...continued

Debugging tips

Use print statements

```
1  def find_match(text, rules):
2      for rule in rules:
3          match = re.search(rule["pattern"], text)
4          if match:
5              print(f"Matched: {rule['pattern']}")
6              print(f"Groups: {match.groups()}")
7              return rule, match
8      print("No match found!")
9      return None, None
```

Print what patterns match and why. Remove prints when done.

Assignment 1 overview

What you'll build

A complete ELIZA implementation that:

- Reads rules from `instructions.txt`
- Handles pre/post substitutions
- Matches patterns and generates responses
- Maintains a conversation loop

Due date

Friday, January 16 (end of Week 2)

Assignment structure

Part	Weight	Description
Implementation	40%	Working ELIZA chatbot
Conversations	20%	5+ diverse test scenarios
Analysis	25%	ELIZA effect, pattern analysis
Reflection	10%	500-1000 word essay
Presentation	5%	Clean, organized notebook

Submission

Submit as a Google Colaboratory notebook. Make sure it runs without errors!

Getting started

Step 1: Read Weizenbaum

[ELIZA paper \(1966\)](#)

Focus on Section 3 for implementation details.

Step 2: Try online ELIZA

masswerk.at/elizabot

See what responses you should expect.

Step 3: Practice regex

regex101.com

Excellent visualization and testing tool.

Common pitfalls

Watch out for

- **Pattern priority:** Higher rank numbers = higher priority
- **Substitution order:** Pre-subs before matching, post-subs after
- **Case sensitivity:** Normalize case for matching
- **Word boundaries:** Use spaces to avoid partial matches
- **Greedy matching:** `(.*)` vs `(.*?)`

Verify your work

Test with the same inputs as online ELIZA to verify your output.

Suggested timeline

Days	Focus
1-2	Core implementation (pattern matching, substitutions)
3-4	Conversation testing and analysis
5-6	Modern chatbot comparison, write reflection
7	Polish notebook, final testing

Pro tip

Start early! Pattern matching can be tricky to debug.

Key takeaways

1. **ELIZA is simple:** Just string manipulation and pattern matching
2. **Pre/post subs:** Normalize input, fix output pronouns
3. **Pattern priority:** Check specific patterns before general ones
4. **Vibe coding:** Use AI tools to accelerate development
5. **Test incrementally:** Build and test piece by piece

Remember

The magic isn't in the algorithm. It's in how humans interpret the output!

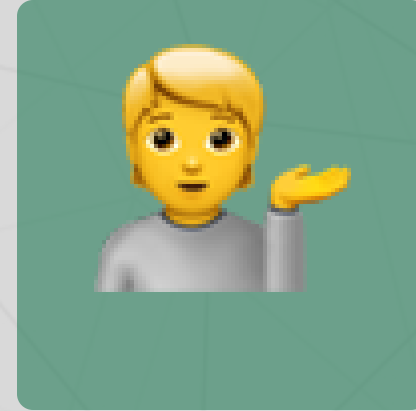
Questions? Want to chat more?



Email me



Join our Discord



Come to office hours

Get help

Start Assignment 1 early. Come to office hours if you get stuck!